4 November – finishing Phase 4

stor01 and GLPI · 15:35 – 18:15 (Germany) · the final file for today, to revise

#	What	Result
1	Step 4 – permanent limits for archive-worker on stor01 (set-property → drop-in files)	✅ **proven after reboot**
2	Incident drill – "deleted the file, but the disk is still full" (lsof +L1)	✅ **500 MB freed**
3	GLPI Incident ticket #4 for the drill	✅ **solved and closed**
4	GLPI L3 Problem behind ticket #3 – archive worker without limits	✅ **solved with Step 4**
5	VMware snapshot 50-processes on all 4 VMs	✅ **Phase 4 complete**

Today in one paragraph: The archive worker on stor01 now has permanent limits – CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 – written by `systemctl set-property` (without --runtime) as drop-in files on the disk, so they came back after a reboot. In the incident drill a deleted 500 MB file stayed on the disk because a running process still held it open: df showed it, du did not, `lsof +L1` found PID 5468, and stopping it freed the space. Both went into GLPI the ITIL way: Incident ticket #4 for the drill (fix it now) and an L3 Problem for the root cause behind ticket #3 (fix it for good), and a snapshot "50-processes" closes Phase 4.

Inside: **Part 1** Step 4 file (with a refresher of tests 1–3) · **Part 2** incident drill · **Part 3** GLPI ticket, Problem, snapshot · **Part 4** Step 4 explained: CPUQuota in time, NFS/ backup sharing, TasksMax vs restart limits.

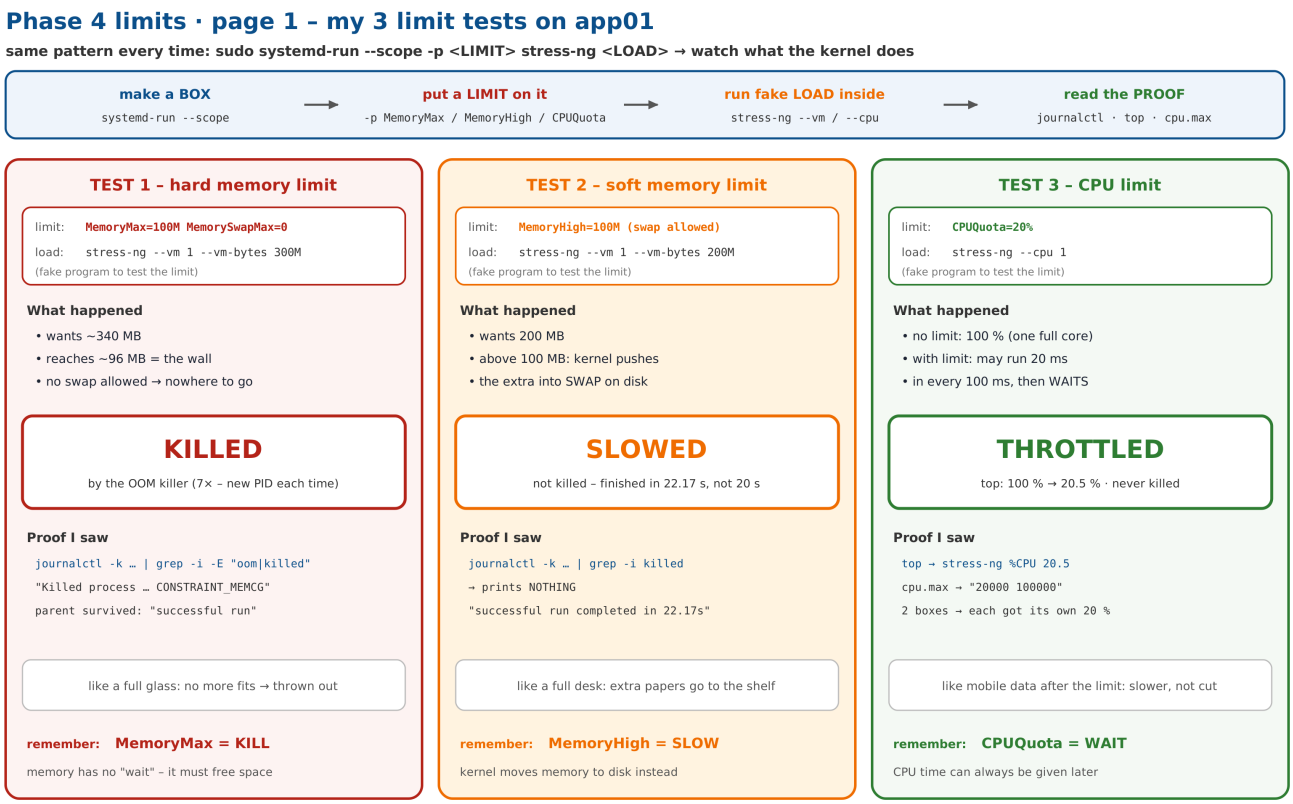
Step 4 on stor01 – permanent limits for archive-worker

Phase 4 · 4 November 2026 · every command with its own screenshot · refresher of tests 1–3 first

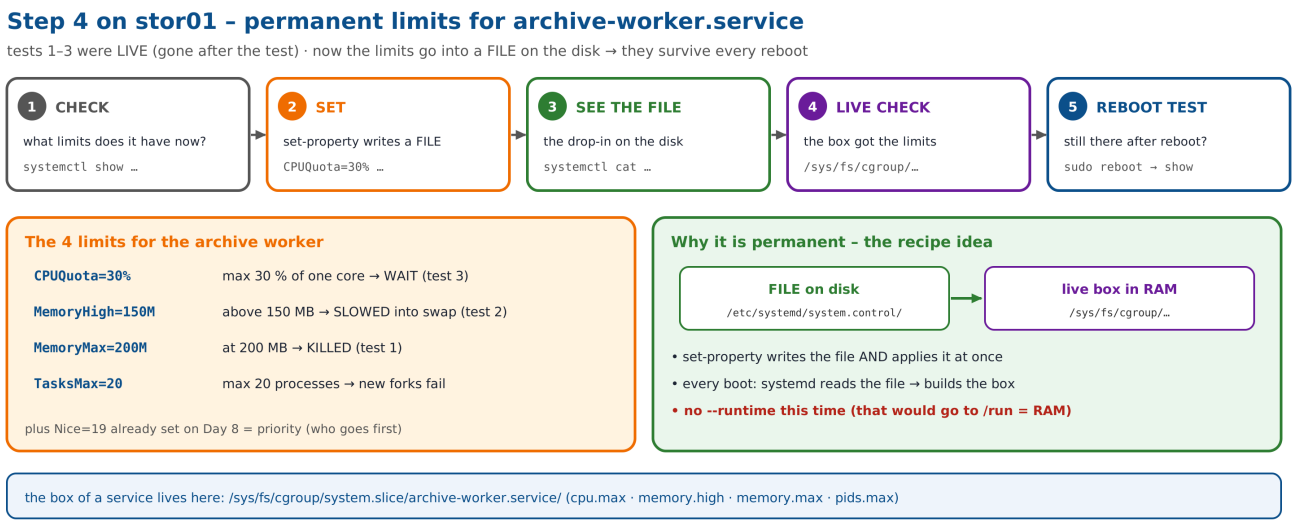
Refresher – what tests 1–3 taught (read this first)

	limit	what happens	remember
Test 1	`MemoryMax=100M`	hard wall – the OOM killer stops the job inside the box	**KILL**
Test 2	`MemoryHigh=100M`	soft limit – extra memory pushed to swap, job slower, not killed	**SLOW**
Test 3	`CPUQuota=20%`	box may run 20 ms of every 100 ms, then waits – throttled	**WAIT**

Tests 1–3 used `systemd-run --scope` = a **temporary** box, a **live change** in RAM – gone when the test ended. Today the limits go on a **real service** and into a **file on the disk** – the "recipe" that systemd reads at every boot. That is what makes them permanent.



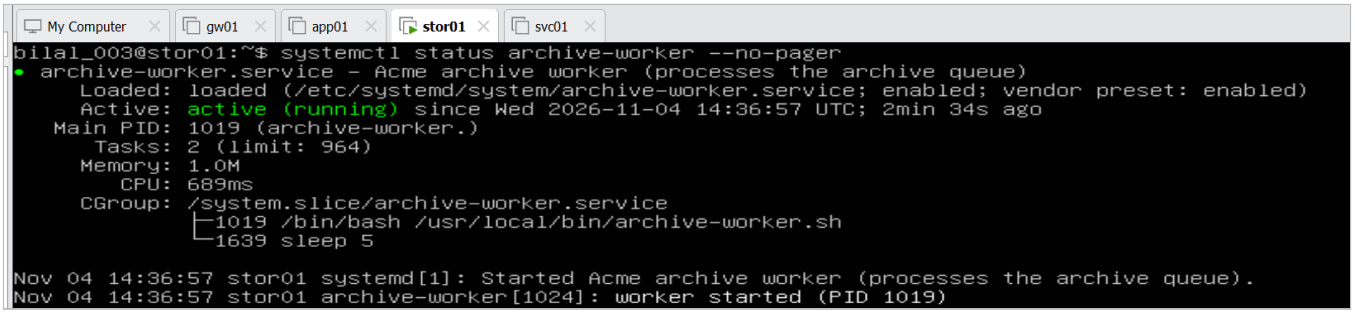
Today's plan – 5 steps



Final paragraph: Tests 1 to 3 were live changes – the temporary boxes and their limits disappeared when each test ended. For a real service the limits must survive a reboot, so they go into a file. sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20 does two things at once: it applies the limits to the running box and writes them as a drop-in file under /etc/systemd/system.control/ on the disk. At every boot systemd reads that file and builds the box again with the same limits. I check the file with systemctl cat, the live box under /sys/fs/cgroup/system.slice/archive-worker.service/, and prove it is permanent with a reboot. CPUQuota makes the worker wait, MemoryHigh slows it, MemoryMax kills it, TasksMax stops it creating too many processes – and Nice=19 from Day 8 still decides that it goes last.

Step 1 – check what the worker has now

```
systemctl status archive-worker --no-pager
```



What it shows: the worker is **active (running)** since boot (enabled = starts automatically). Its box is `/system.slice/archive-worker.service` with 2 processes inside: the worker script (PID 1019) and the `sleep 5` it runs while waiting for jobs. **Tasks: 2 (limit: 964)** – 964 is the default, not a limit I chose. Memory 1.0 MB – tiny.

```
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
```



Before: `infinity` = no CPU or memory limit · TasksMax 964 = the default · `Nice=19` = the low priority I set on Day 8. Priority says *who goes first*; it does not cap anything.

Step 2 – set the 4 limits, permanently

```
sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M
MemoryMax=200M TasksMax=20
```



No output = success. **No `--runtime`** this time – with it the limits would go to `/run` (RAM) and vanish at reboot. Without it, set-property does two things at once: **applies** the limits to the running box and **writes** them to files on the disk.

limit	meaning for the worker
`CPUQuota=30%`	max 30 % of one core – a big archive job cannot take the CPU from NFS or backups (test 3 = wait)
`MemoryHigh=150M`	above 150 MB it is slowed into swap (test 2)
`MemoryMax=200M`	at 200 MB the OOM killer stops it (test 1) – `Restart=on-failure` in its unit file then starts it again
`TasksMax=20`	max 20 processes – a bug that forks endlessly cannot fill the PID pool

Step 3 – see the files on the disk (the recipe)

```
systemctl cat archive-worker
```

```
bilal_003@stor01:~$ systemctl cat archive-worker
# /etc/systemd/system/archive-worker.service
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
```

Part 1 – my original unit file `/etc/systemd/system/archive-worker.service` from Day 8 – unchanged: the script, `Restart=on-failure`, `Nice=19`, `IOSchedulingClass=idle`.

```
# /etc/systemd/system.control/archive-worker.service.d/50-CPUQuota.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
CPUQuota=30%

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryHigh.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryHigh=157286400

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryMax=209715200

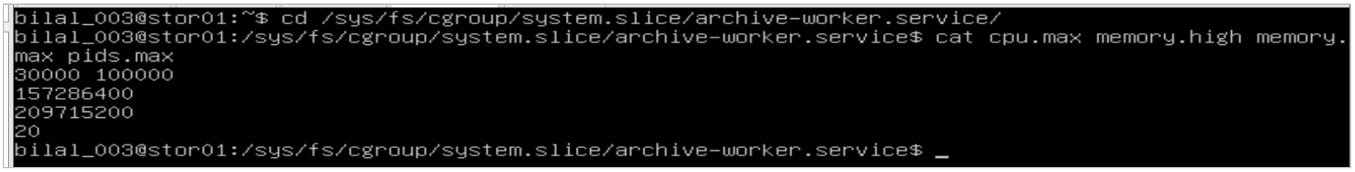
# /etc/systemd/system.control/archive-worker.service.d/50-TasksMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
TasksMax=20
bilal_003@stor01:~$ _
```

Part 2 – four new drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, one per limit (`50-CPUQuota.conf` …). They are added *on top of* the unit file without changing it. "Do not edit" – to change a limit, run set-property again.

Memory is stored in **bytes**: 157286400 = 150 × 1024 × 1024 = 150 MB · 209715200 = 200 MB.

Step 4 – check the live box in the kernel

```
cd /sys/fs/cgroup/system.slice/archive-worker.service/
cat cpu.max memory.high memory.max pids.max
```

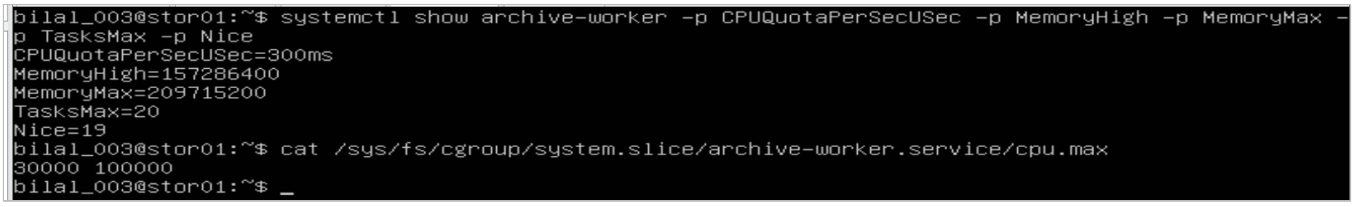


file	value	meaning
`cpu.max`	30000 100000	30 ms of every 100 ms = 30 %
`memory.high`	157286400	150 MB soft limit
`memory.max`	209715200	200 MB hard wall
`pids.max`	20	max 20 processes

The file on the disk (step 3) is the **recipe**; these values in `/sys/fs/cgroup` are the **live box** in RAM that the kernel enforces.

Step 5 – the reboot test (proof that it is permanent)

```
sudo reboot
# after logging in again:
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
cat /sys/fs/cgroup/system.slice/archive-worker.service/cpu.max
```



after reboot	meaning
`CPUQuotaPerSecUSec=300ms`	30 % = 300 ms of CPU per second – same limit, new way of writing it
`MemoryHigh=157286400` · `MemoryMax=209715200`	150 MB soft · 200 MB hard – still there
`TasksMax=20` · `Nice=19`	process cap and priority – still there
`cpu.max 30000 100000`	the kernel's live box was rebuilt with the limit

Proof: RAM was emptied by the reboot, yet the box came back with every limit – because systemd read the drop-in files on the disk at boot and built the box again. ✅ Permanent.

Final paragraph: Tests 1 to 3 were live changes – their temporary boxes and limits disappeared when each test ended. For the real archive worker on stor01 the limits must survive a reboot, so they go into files. Before, the worker had no CPU or memory limit

(`infinity`), the default TasksMax 964 and Nice=19 from Day 8. `sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20` – without `--runtime` – applied the limits at once and wrote four drop-in files under `/etc/systemd/system.control/archive-worker.service.d/`, next to my unchanged unit file; `systemctl cat` shows both. The kernel's live box in `/sys/fs/cgroup/system.slice/archive-worker.service/` confirms `cpu.max 30000 100000`, memory.high 150 MB, memory.max 200 MB and pids.max 20. Now the worker waits above 30 % CPU, is slowed above 150 MB, is killed at 200 MB – and restarted by `Restart=on-failure` – and can never run more than 20 processes, while Nice=19 still makes it go last. At every boot systemd reads the drop-ins and builds the box again: the reboot test proved it: after the restart `systemctl show` still reported 300ms, 150 MB, 200 MB, 20 tasks and Nice=19, and cpu.max was again 30000 100000.

Step 4 explained – how the 4 limits really work

questions from revising: how does CPUQuota=30% work in time? what does TasksMax count? how are restarts limited?

CPU throttling – CPUQuota=30% on stor01, everything on one page

one CPU, three jobs: the archive worker · the NFS share for app01 · the nightly backup (or any other job running in the background)

① Who gets the CPU

WITHOUT a limit



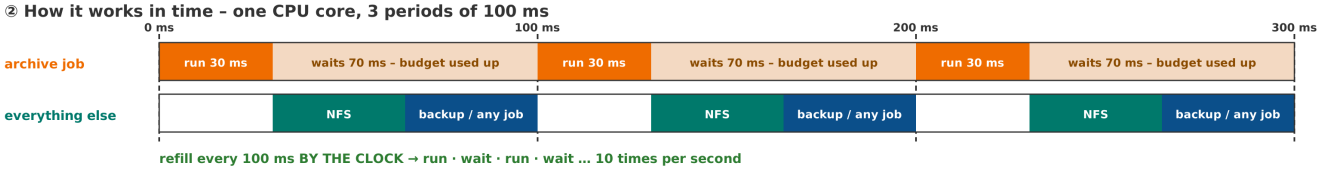
the archive job takes ~92 %
app01 waits for NFS · the backup crawls
the others only get tiny gaps

WITH CPUQuota=30%



the archive job never gets more than 30 %
NFS and the backup keep their share
the archive job just takes longer

② How it works in time – one CPU core, 3 periods of 100 ms



refill every 100 ms BY THE CLOCK → run · wait · run · wait … 10 times per second

③ The facts to remember

`cpu.max = 30000 100000`
30 000 µs (30 ms) budget in every 100 000 µs (100 ms) period
runs at FULL speed
until its 30 ms are used up – the CPU never slows down
then it WAITS
only until the next 100 ms period – not until the backup ends
the 70 ms go to
whatever else needs the CPU: NFS, backup, sshd, any job
nothing else running?
the CPU stays idle – CPUQuota is a HARD cap (30 % stays 30 %)
never killed
CPU can always be given later – too much CPU only waits

④ Pocket money that refills every "day"

every day you get 300 (= 30 ms every 100 ms)
you spend at your NORMAL speed until it is gone
then you have 0 → you wait until tomorrow
tomorrow: 300 again → spend → wait → …



day 1 = 100 ms | day 2 = 100 ms | day 3 = 100 ms

the speed of spending never changes – only the waiting

⑤ Proven in test 3 (app01): no limit → stress-ng 100 % · CPUQuota=20% → 20.5 % · cpu.max 20000 100000 · never killed

and on stor01 (Step 4): archive-worker cpu.max 30000 100000 – still there after reboot

Final paragraph: On stor01 the archive worker, the NFS share for app01 and the nightly backup share one CPU. Without a limit one big archive job could take about 92 % of it, so app01 would wait for NFS and the backup would crawl. CPUQuota=30% gives the archive box a budget of 30 ms in every 100 ms period (cpu.max 30000 100000). The job runs at its normal full speed until the 30 ms are used up, then it must wait – not until the backup ends, only until the next 100 ms period, when the clock refills its budget. In the other 70 ms the CPU works for whatever else needs it: NFS, the backup or any other job; if nothing else is running the CPU stays idle, because CPUQuota is a hard cap. Like pocket money paid every day, the spending speed never changes – only the waiting. This repeats 10 times per second, so the archive job gets 30 % on average and simply takes longer; nothing is killed. Test 3 proved it (100 % → 20.5 % with 20 %), and on stor01 the 30 % limit survived a reboot.

Two lines made clear – who shares stor01's CPU, and what TasksMax really counts

① "A big archive job cannot take the CPU from NFS and backups"

NFS (not NFC) = the network share on stor01 · all three jobs run ON stor01 and share ITS CPU – it is not about the CPU of another server

WITHOUT a limit

stor01 CPU (100 %)



the archive job takes almost everything
→ app01 waits for the NFS share (slow)
→ the nightly backup runs very slowly
(backup + NFS fight for the last few %)

WITH CPUQuota=30%

stor01 CPU (100 %)



the archive job may use at most 30 %
→ NFS keeps answering app01 quickly
→ the backup still gets its share
(the archive job just takes longer – fine)

② TasksMax is NOT "how many restarts" – it is "how many processes at the SAME time"

TasksMax=20

max 20 processes/threads ALIVE at once in the box

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20

the 21st fork fails: "Resource temporarily unavailable"
protects stor01's PID pool from a fork bug

restarts – a DIFFERENT setting

`Restart=on-failure` → restart it when it crashes / is killed
`StartLimitBurst=5` → at most 5 starts …
`StartLimitIntervalSec=10s` … within 10 seconds

more often → systemd gives up: "start request repeated too quickly" – the service stays failed

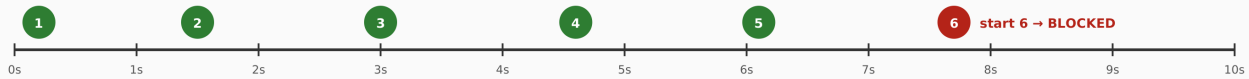
(5 in 10 s are the systemd defaults)

Final paragraph: stor01 does three jobs on the same CPU – it shares /backups over NFS to app01, it runs the nightly backups, and it runs the archive worker. Without a limit one big archive job could take almost all of stor01's CPU, so app01 would wait for the NFS share and the backups would crawl. CPUQuota=30% lets the worker use at most 30 % of a core, so NFS and the backups always keep their share; the archive job only takes longer. TasksMax=20 is something else: it is the maximum number of processes or threads alive in the worker's box at the same time – the 21st fork fails, which protects the PID pool. It does not count restarts. Restarts are set by Restart=on-failure (restart when it dies) and limited by StartLimitBurst and StartLimitIntervalSec (by default 5 starts in 10 seconds, then systemd gives up and the service stays failed).

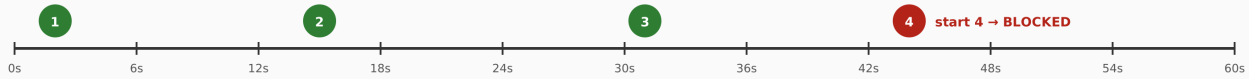
StartLimitBurst + StartLimitIntervalSec – the brake on endless RESTARTS (not threads)

one rule: "at most BURST starts within INTERVAL seconds" – two examples with different numbers

Example A – the DEFAULT (what every service has if I set nothing): 5 starts in 10 seconds



Example B – MY OWN choice in the unit file: 3 starts in 60 seconds



Example B written in the unit file

```
[Unit]
StartLimitIntervalSec=60 ← INTERVAL: 60 s
StartLimitBurst=3 ← BURST: 3 starts

[Service]
Restart=on-failure ← restart when it dies
```

Check and recover

```
systemctl show archive-worker -p StartLimitBurst \
  -p StartLimitIntervalUSec
```

archive-worker has nothing set → shows the default 5 · 10s

blocked? fix the cause, then:

```
sudo systemctl reset-failed archive-worker
```

Final paragraph: Restart=on-failure starts a service again when it dies. To stop an endless crash–restart loop, systemd counts the STARTS (not threads or processes) inside a time window: StartLimitBurst is how many starts are allowed, StartLimitIntervalSec is the length of the window. If I set nothing, the default is 5 starts in 10 seconds (example A); in the unit file I can choose my own numbers, for example StartLimitIntervalSec=60 and StartLimitBurst=3 = at most 3 starts per minute (example B). One start too many is blocked with "start request repeated too quickly" and the service stays failed until I fix the cause and run systemctl reset-failed. TasksMax is different: it counts processes running at the same moment.

Incident drill on stor01 – "I deleted the file, but the disk is still full"

Phase 4 · 4 November 2026 · a deleted file still held open by a running program · every command with its own marked-up screenshot

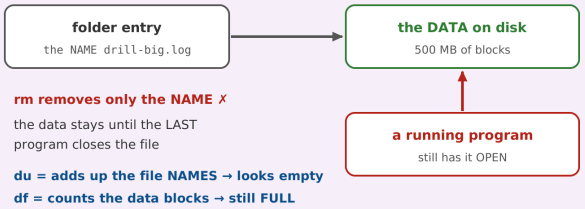
Incident drill on stor01 – "I deleted the file, but the disk is still full"

a classic real ticket and interview question: a program still holds a deleted file open, so its space is NOT freed

Ticket (client words): "/backups on stor01 was nearly full. I deleted a big old log file – but df still says it is full!"

real-life cause: logrotate or a person deletes a log file while the program that writes it keeps running

Why the space is not freed



rm removes only the NAME ✗

the data stays until the LAST program closes the file

du = adds up the file NAMES → looks empty

df = counts the data blocks → still FULL

The drill – 4 parts

1 BUILD the problem
make a 500 MB file · a program holds it · delete it

2 SEE the symptom
df: still used · du: nothing there

3 FIND the cause
sudo lsof +L1 → "(deleted)" + the PID

4 FIX + TICKET
stop that program → space free · GLPI ticket

How admins fix it in real life

- best: restart the service that holds the file (systemctl restart ...) – it closes the old file and opens a new one
- if it must not restart: empty the deleted file through the process: : > /proc/<PID>/fd/<FD> – frees the space at once

Final paragraph: In Linux, deleting a file with rm only removes its name from the folder. The data blocks are freed only when the last program that has the file open closes it. So if a running program still holds a deleted file – typically a log file removed while the service keeps writing – df still counts the space as used, while du, which adds up the file names it can see, shows nothing. The tool to find it is sudo lsof +L1: it lists open files whose name is gone, marked "(deleted)", with the PID and file descriptor of the program holding them. The fix is to restart or stop that program (or empty the file through /proc/<PID>/fd/<FD> if it must keep running), and the space comes back at once. In the drill I build this on stor01's /backups, find it, fix it and record it as a GLPI ticket.

The incident drill – final corrected version, in plain words

an INCIDENT drill (a disk-space mystery), not a restore drill · /backups was used only because it is a separate disk with easy numbers

1 look at the disk — `df -h /backups`
how full is the /backups disk? → 60K used

2 make a big file — `sudo fallocate -l 500M …`
fallocate = FILE ALLOCATE = reserve 500 MB for one empty file (no stress, just space) → 501M used

3 a process holds it OPEN — `sleep 3600 < file &`
the KEY step: process sleep (PID 5468) holds the file open on its file descriptor 0 – it plays a service and its log

4 delete the name — `sudo rm file`
only the NAME is gone – USED stays 501M, because the process still holds the file open

5 prove the mystery — `sudo du -sh /backups`
du = disk usage · -s = summary (one total) · -h = human-readable → only 56K (du counts names)

6 find + fix — `sudo lsof +L1 → kill 5468`
lsof = list open files · +L1 = less than 1 name left = deleted but still open → shows the PID of the PROCESS → stop it → 60K

Words – say it right

fallocate	file ALLOCATE (reserve), not "locate"
the PROCESS has the PID	the file has no PID
process HOLDS it open	a file has no "parent" – that word is for processes
df vs du	df counts disk blocks · du counts file names

Same effect with a copy A → B

cp is reading file X · someone deletes X
→ cp still finishes the copy · X's space stays used
→ freed when cp closes X – after seconds, nobody notices

a SERVICE keeps its log open for weeks
→ the space stays "lost" until the service is restarted

Final paragraph: This was an incident drill, not a restore drill. df -h /backups showed the disk almost empty (60K). sudo fallocate (file allocate) reserved 500 MB for one empty file – no stress, just space – so used went to 501M. The key step: sleep 3600 < file & – a process (PID 5468) holding the file open on its file descriptor 0, like a service holds its log. sudo rm removed only the name, so used stayed at 501M, while du -sh (disk usage, summary, human-readable) counted only 56K, because du adds up the names it can find and df counts the blocks. lsof +L1 (list open files with fewer than one name) showed the deleted file and the PID of the process holding it – the file itself has no PID and no parent. Stopping that process freed the 500 MB; on a real server I would restart the service. The same happens when a file is deleted while cp is copying it, but a copy closes the file within seconds – a service keeps its log open for weeks, which is why this becomes a real incident.

1 – Build the problem

```
df -h /backups
```



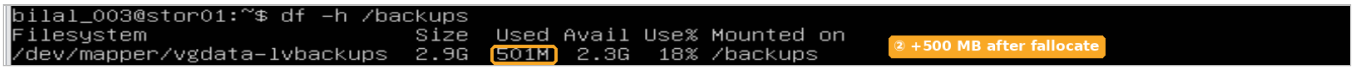
① **Before:** /backups (an LVM volume, 2.9 GB) uses only **60K** – it is empty, 1 %.

```
sudo fallocate -l 500M /backups/drill-big.log
```



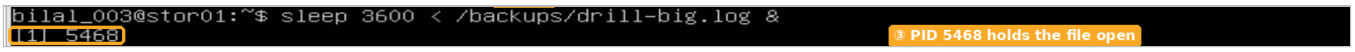
No output = success. `fallocate` reserves 500 MB on the disk at once – our big "log file".

```
df -h /backups
```



② **After fallocate: 501M** used, 18 % – the new file takes the 500 MB.

```
sleep 3600 < /backups/drill-big.log &
```



③ **A program holds the file open:** `sleep` uses the file as its input and runs for an hour in the background – job **[1]**, **PID 5468**. It stands in for a real service keeping its log file open.

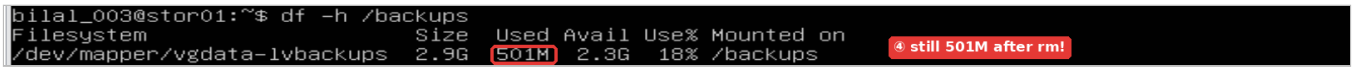
```
sudo rm /backups/drill-big.log
```



No output = the file name is deleted.

2 – See the symptom

```
df -h /backups
```



④ **The problem:** still **501M** used after `rm`! `df` counts the data blocks on the disk – they are still there.

```
sudo du -sh /backups
```



⑤ **The contradiction:** `du` adds up the file names it can find – the name is gone, so it sees only **56K**. When df and du disagree, think "deleted but still open".

3 – Find the cause

```
sudo lsof +L1
```



⑥ **The cause:** `lsof +L1` lists open files that have no name left (fewer than 1 link). Marked: **PID 5468** (sleep) · **524288000** bytes = 500 MB · **(deleted)**. FD `0r` = its input, opened for reading · NLINK `0` = no names left.

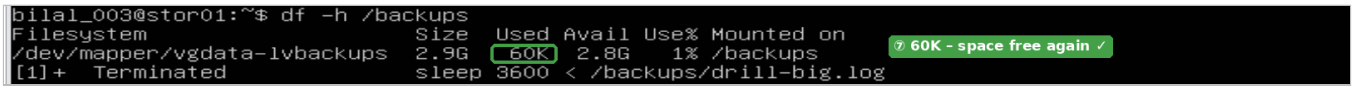
4 – Fix it

```
kill 5468
```



Stop the program that holds the file (the PID from lsof).

```
df -h /backups
```



⑦ **Fixed:** back to **60K**, 1 % – the kernel freed the 500 MB as soon as the last program closed the file. `[1]+ Terminated` confirms the background job ended.
On a real server I would not kill a service blindly – I would **restart** it (`sudo systemctl restart <service>`) so it closes the old log and opens a new one; if it must keep running, `: > /proc/<PID>/fd/<FD>` empties the deleted file in place.

5 – Record it as a GLPI ticket (text ready to paste)

Title: stor01 – /backups still full after deleting a large log file
Type: Incident · **Category:** Storage · **Urgency:** Medium · **Asset:** stor01
Description: User deleted /backups/drill-big.log (500 MB) but `df -h /backups` still shows 501M used (18 %). `du -sh /backups` shows only 56K.
Diagnosis: `sudo lsof +L1` → process `sleep`, PID 5468, holds /backups/drill-big.log (deleted), 524288000 bytes, NLINK 0. The space is not freed while a process keeps the deleted file open.
Solution: Stopped the holding process (kill 5468). `df -h /backups` → 60K used, 1 %. In production: restart the service that holds the file, or empty it via /proc/<PID>/fd/<FD>.
Prevention: rotate logs with logrotate using `copytruncate` or a service reload, never delete open log files by hand; check `lsof +L1` when df and du disagree.

Final paragraph: This was an incident drill, not a restore drill. `df -h /backups` showed the disk almost empty (60K). `sudo fallocate` (file allocate) reserved 500 MB for one empty file – no stress, just space – so used went to 501M. The key step: `sleep 3600 < file &` – a process (PID 5468) holding the file open on its file descriptor 0, like a service holds its log. `sudo rm` removed only the name, so used stayed at 501M, while `du -sh` (disk usage, summary, human-readable) counted only 56K, because du adds up the names it can find and df counts the blocks. `lsof +L1` (list open files with fewer than one name) showed the deleted file and the PID of the process holding it – the file itself has no PID and no parent. Stopping that process freed the 500 MB; on a real server I would restart the service, and prevent it with proper log rotation. The same happens when a file is deleted while `cp` is copying it, but a copy closes the file within seconds – a service keeps its log open for weeks, which is why this becomes a real incident. The incident is recorded in GLPI as ticket #4.

Closing Phase 4 – Incident ticket, L3 Problem, snapshot

4 November 2026 · GLPI on svc01 (http://192.168.60.40) · ITIL: Incident = fix it now · Problem = find the root cause, fix it for good

Finishing Phase 4 – incident ticket, L3 Problem, snapshot

ITIL in one line: an INCIDENT = one thing broke, fix it now · a PROBLEM = why does it keep happening? find the root cause, fix it for good

A · Ticket – Incident

today's drill on stor01

"/backups still full after deleting a large log file"

L1/L2 fixes it → status Solved

text ready in the drill file

B · Problem – L3 Engineering

root cause behind ticket #3 (Day 8):

the archive worker had no limits – one big job could starve stor01

solution = Step 4 permanent limits

→ status Solved

C · Snapshot "50-processes"

VMware, on all 4 VMs:

`gw01 · app01 · stor01 · svc01`

a safe point to return to before Phase 5

→ Phase 4 complete

How they link in GLPI

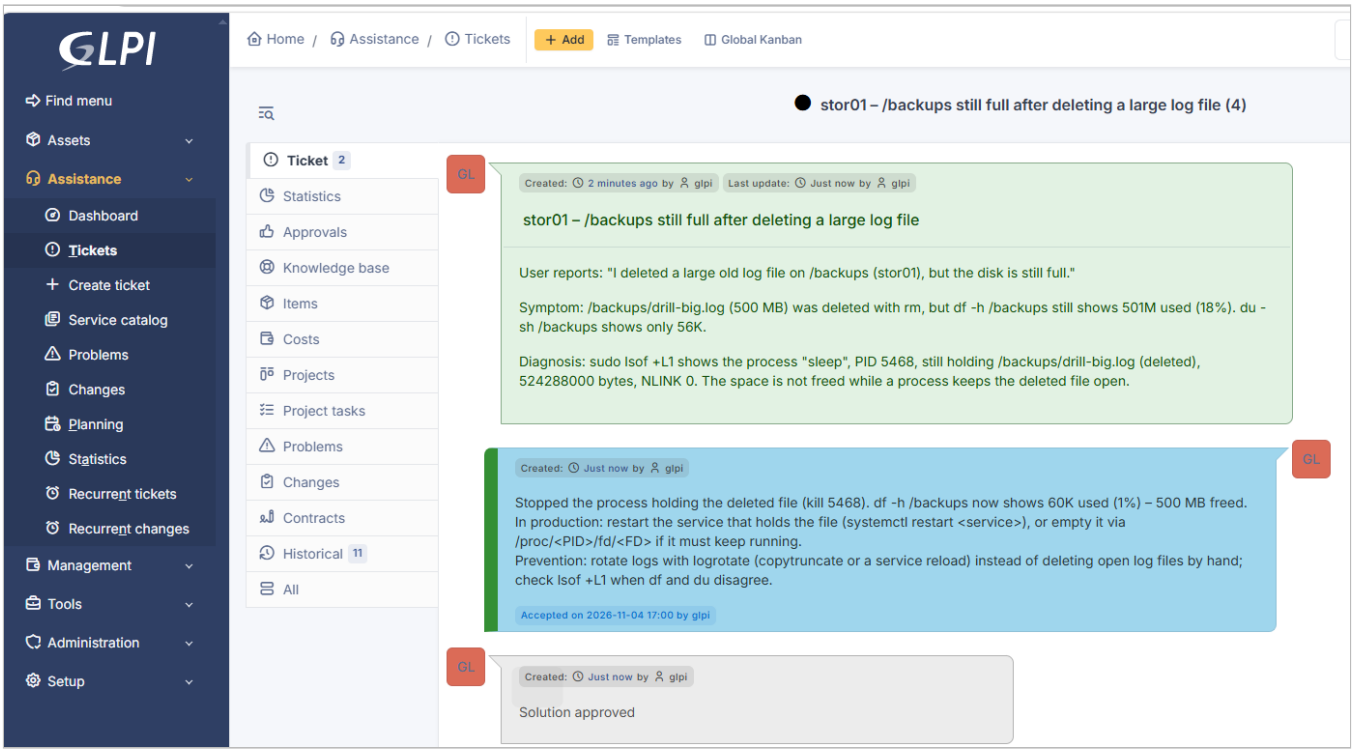
ticket #3 (archive queue, Day 8, L1 → L2) —linked to→ Problem "archive worker without resource limits" (group L3 – Engineering)

the Problem records: symptoms · cause · impact · solution (Step 4) – so the next admin sees WHY and WHAT was changed

Final paragraph: Phase 4 ends with the service-desk side and a safe point. A – today's drill becomes an Incident ticket on stor01: symptom, diagnosis with lsof +L1, solution and prevention, then Solved. B – an Incident is one event; a Problem asks why it keeps happening. The L3 Problem "archive worker without resource limits" is linked to ticket #3 from Day 8, assigned to the group L3 – Engineering, and records the root cause (no limits, so one big archive job could take stor01's CPU and memory) and the permanent solution from Step 4 (CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20, proven after reboot) – then it is Solved. C – a VMware snapshot "50-processes" of all four VMs is a safe point to return to before Phase 5.

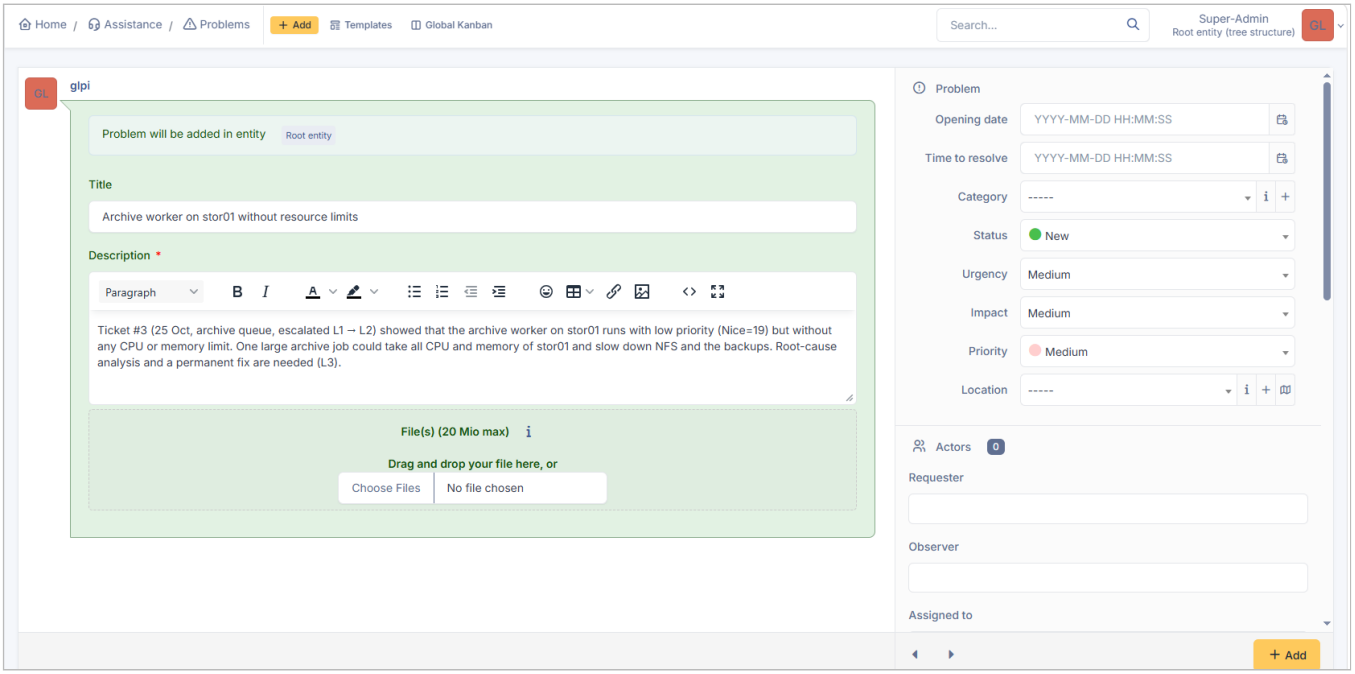
A – Incident ticket #4 for today's drill – Solved and closed

Title *stor01 – /backups still full after deleting a large log file* · Type Incident · Category Storage & Backup · Requester **alice** (Requester = who reported it) · Assigned to group **L2 – Linux Storage** · Item **stor01** (Item = what is broken). The description holds the symptom and the lsof diagnosis; the solution holds the fix and the prevention. The solution was accepted and the ticket closed (black dot). In a company alice would approve it herself.

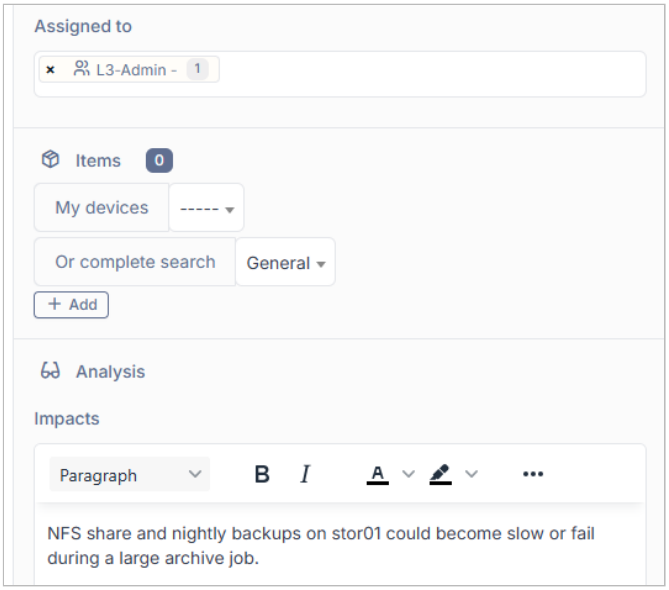


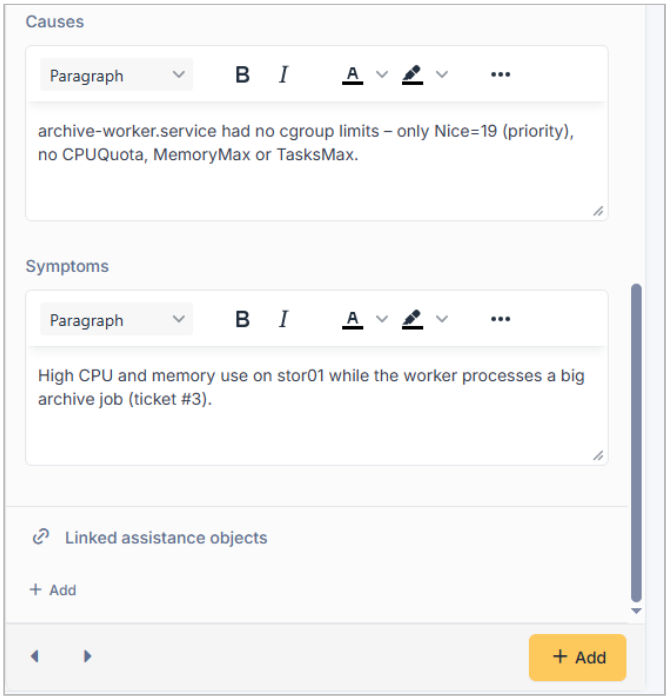
B – L3 Problem: "Archive worker on stor01 without resource limits"

An Incident is one event; a Problem asks why it happens. This Problem is the root cause behind **ticket #3** (25 Oct, archive queue, escalated L1 → L2). Assigned to group **L3-Admin**. Note: it must be created under **Assistance → Problems** – the same text under Tickets would create a ticket.



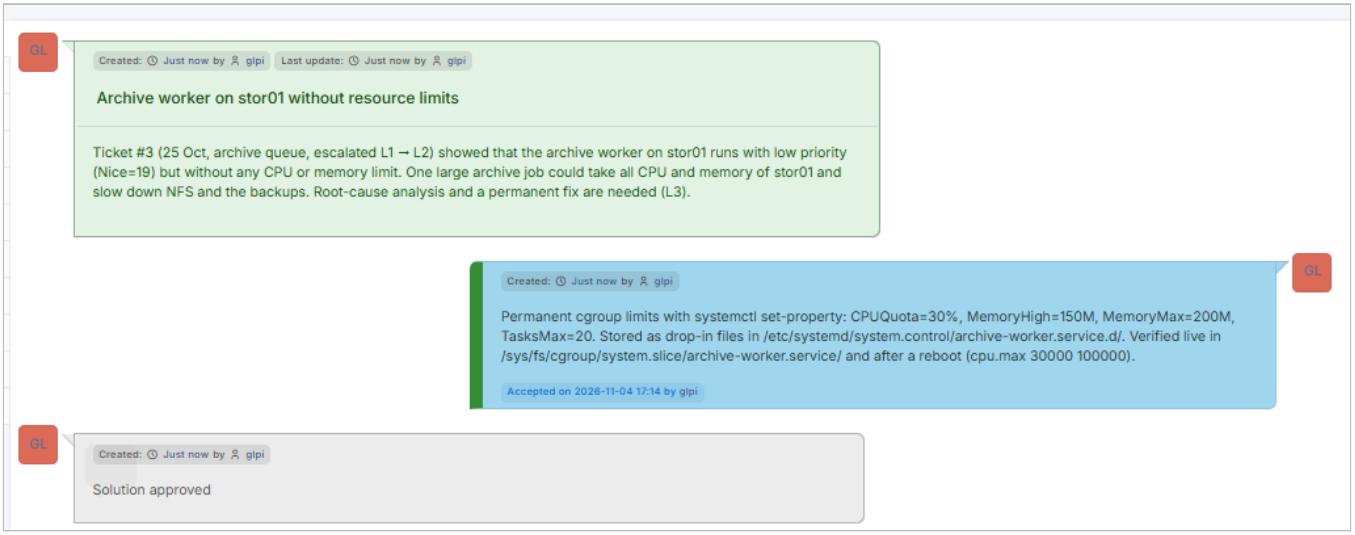
Analysis – the three boxes are part of the Problem form:





Impacts: NFS and backups could become slow · Causes: no cgroup limits, only Nice=19 · Symptoms: high CPU and memory during a big archive job.

B – Solution: the Step 4 limits – approved



The solution records exactly what was changed and how it was verified: `CPUQuota=30%`, `MemoryHigh=150M`, `MemoryMax=200M`, `TasksMax=20`, drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, checked in `/sys/fs/cgroup` and after a reboot. Ticket #3 and stor01 are linked to the Problem.

C – Snapshot "50-processes"

VMware snapshot **50-processes** on all four VMs (gw01, app01, stor01, svc01) – description *Phase 4 complete: processes, signals, limits, incident drill, L3 Problem*. A safe point to return to before Phase 5.

Final paragraph: Phase 4 ends on the service-desk side. Today's drill became Incident ticket #4: requester alice, group L2 – Linux Storage, item stor01, with the symptom (df 501M, du 56K), the diagnosis (lsof +L1, PID 5468, deleted file still open), the solution and the prevention – solved, approved and closed. Behind ticket #3 from Day 8 I opened an L3 Problem, "Archive worker on stor01 without resource limits", assigned to L3-Admin, with its analysis – impact on NFS and backups, cause no cgroup limits (only Nice=19), symptom high CPU and memory during big jobs – and the permanent solution from Step 4: CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 as drop-in files, verified after a reboot. An Incident fixes one event; a Problem removes the cause so it does not come back. A snapshot "50-processes" of all four VMs closes Phase 4.

Before Phase 5 (logging) – the 6 things from Phases 1–4 you will use again

5-minute revision · left = what you already know · right = where it comes back in Phase 5 (journald · rsyslog · logrotate)

1 systemd services — Phase 3

- every program runs as a .service unit
- systemctl status / start / restart / enable
- unit file + drop-ins in /etc/systemd/system/

→ systemd-journald and rsyslog ARE services · each service's output lands in the journal → journalctl -u nginx

2 journalctl — Phases 3–4

- journalctl -u <service> = one service's log
- journalctl -k = kernel messages (the OOM kills!)
- --since "5 min ago" · | grep -i killed · | tail -5

→ the main tool of Phase 5 – filter by service, time, priority (-p err), boot (-b), follow live (-f)

3 RAM vs disk – live vs permanent — Phase 4

- /run = RAM → gone at reboot
- /var = disk → stays
- a setting is permanent only if it is in a FILE on the disk

→ journal in /run/log/journal = lost at reboot · in /var/log/journal = kept → journald.conf Storage=persistent

4 deleted file still open — Phase 4 drill

- rm removes only the name
- a service holding its log keeps the space
- df vs du · lsof +L1 · restart the service

→ exactly why logrotate exists: rotate logs safely (copytruncate or a service reload) instead of deleting them

5 disks and space — Phases 2–3

- df -h · du -sh · LVM volumes
- disk-full alert timer from Phase 3
- /var/log can fill a disk

→ logs grow forever → logrotate (daily, rotate 7, compress) + journald limits (SystemMaxUse=)

6 network + the 4 servers — Phase 1

- gw01 .1 · app01 .10 · stor01 .20 · svc01 .40
- ports, firewall (iptables) on gw01
- SSH between the servers

→ rsyslog sends logs from app01/gw01/svc01 to stor01 = the central log server (UDP/TCP port 514)

Where Phase 5 is going – logs from every server end up on stor01



Final paragraph: Phase 5 is about logs, and it builds on six things I already know. Services (Phase 3): journald and rsyslog are services, and every service writes into the journal. journalctl (Phases 3–4): -u for one service, -k for the kernel like the OOM kills, with --since, grep and tail. RAM vs disk (Phase 4): a journal in /run is lost at reboot, one in /var/log/journal is kept – permanent only when written to the disk. The deleted-but-open file (Phase 4 drill): the reason logrotate rotates logs safely instead of deleting them. Disks and space (Phases 2–3): logs grow forever, so they need rotation, compression and size limits. Network and the four servers (Phase 1): rsyslog will send logs from gw01, app01 and svc01 over port 514 to stor01, the central log server.